

ระบบตรวจจับและแยกเกรดตำหนิบนผิวมะม่วงแบบเรียลไทม์

สรุปการออกแบบ Data Pipeline ตั้งแต่การเก็บภาพจนถึงการประเมินผลและการนำไปใช้งาน

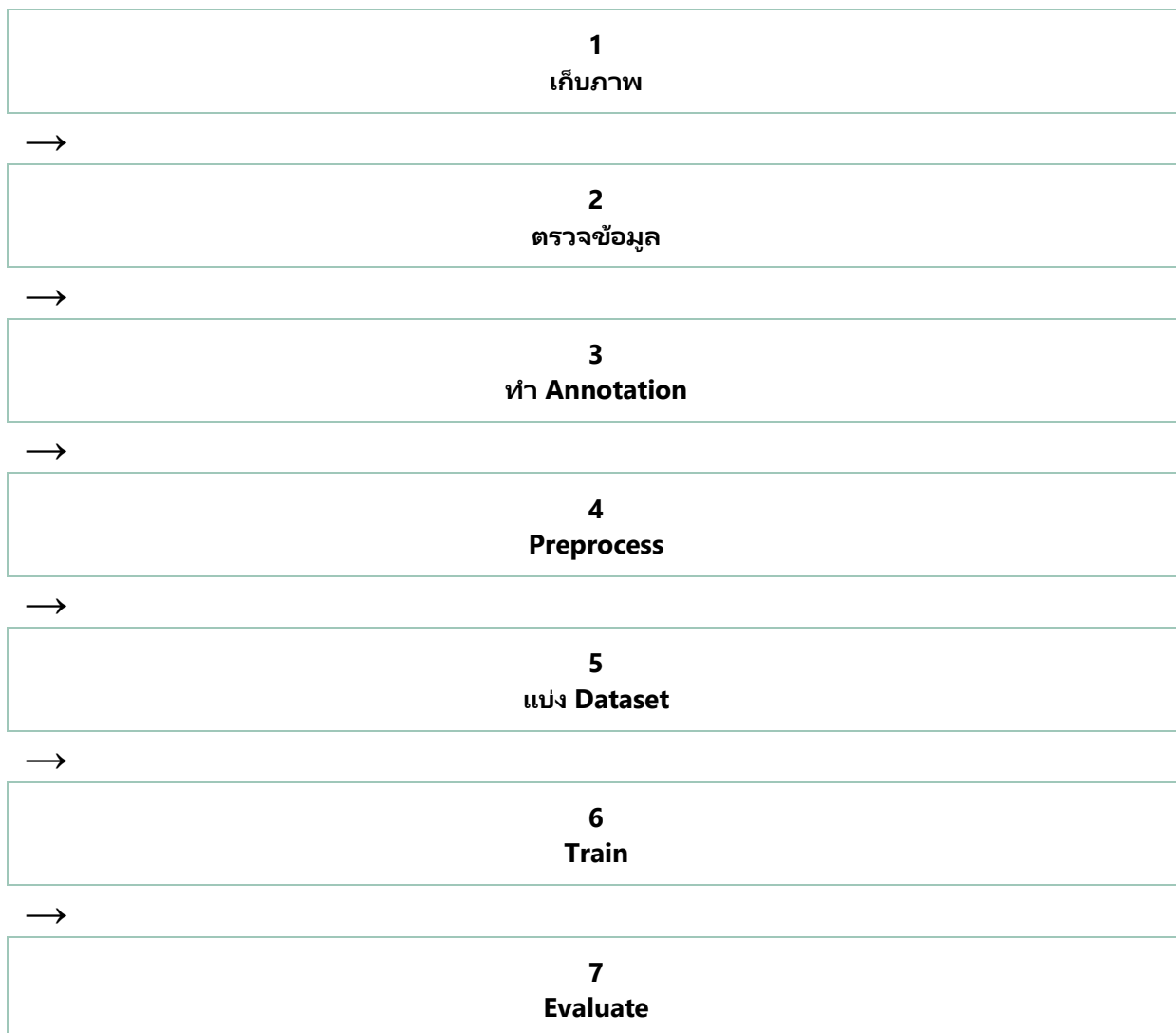
ขอบเขตหลัก

มะม่วง 1 ชนิด · ภาพ 1,000–1,500 ภาพ · 3 เกรด · ตรวจตำแหน่งตำหนิ · ใช้ RGB และ silhouette
เป้าหมาย: ความแม่นยำอย่างน้อย 85% และความเร็วอย่างน้อย 15 FPS

เอกสารสรุปจาก DESIGN.md · ฉบับวันที่ 21 กันยายน 2026

1. ภาพรวม Pipeline

ระบบแบ่งเป็น 8 ขั้นตอน โดยทุกขั้นตอนต้องเก็บ version และ metadata เพื่อให้ผลลัพธ์ตรวจสอบย้อนกลับได้และสามารถรันซ้ำจากข้อมูลชุดเดิมได้





8 Deploy

ภาพและ metadata

ภาพมะม่วงจาก Lighting Chamber, พื้นหลังสีเดียว, มุมกล้องคงที่ พร้อมรหัสลูกมะม่วงและ session การถ่าย

ผลการตรวจ

ตำแหน่งตำหนิ, mask, grade, confidence, latency และ model version

Detection + Grading

YOLOv8-nano ตรวจตำแหน่งตำหนิ และ MobileNetV2 จำแนก 3 เกรด

สามระบบ

HSV baseline, RGB-only และ RGB + silhouette โดยใช้ test manifest เดียวกัน

หลักสำคัญ: แบ่งข้อมูลตาม fruit_id ไม่ใช่สัณฐานภาพ เพื่อป้องกันภาพหลายมุมของมะม่วงลูกเดียวกันรั่วไปอยู่ทั้ง train และ test

2. การเก็บข้อมูลและ Annotation

2.1 การเก็บภาพ

- กำหนดรหัส fruit_id ให้มะม่วงแต่ละลูก และ image_id ให้แต่ละภาพ
- บันทึก session, กล้อง, ระยะ, มุม, resolution และ lighting configuration
- เก็บภาพต้นฉบับไว้ใน data/raw แบบ immutable
- สร้าง checksum เพื่อตรวจ duplicate และยืนยันว่าไฟล์ไม่ถูกเปลี่ยน

2.2 Mask ที่ต้องใช้

Fruit mask	HSV/background subtraction แล้วเลือก contour หลัก เติมพื้นที่และลบ noise	ตัดพื้นหลัง, สร้าง crop และ silhouette
Defect mask	ผู้ annotator วาด polygon ของตำหนิ; ใช้ SAM ช่วยร่างได้	คำนวณพื้นที่ตำหนิ, train/QA และ derive bounding box
Bounding box	คำนวณจากขอบเขตของ defect mask	ใช้เป็น label สำหรับ YOLOv8-nano

2.3 Label ของเกรด

เกรดระดับผลไม้ไม่ต้องมาจากผู้เชี่ยวชาญ: เกรดส่งออก, เกรดบริโภคในประเทศ, ตกเกรด ส่วน defect-area ratio เป็น feature สำหรับวิเคราะห์และสร้าง baseline ไม่ใช่ ground truth เพียงอย่างเดียว

ต้องกำหนดก่อนเริ่ม label จริง: คู่มือขอบเขตตำหนิ, กรณีตำหนิซ้อนหรือถูกบัง, มะม่วงไม่มีตำหนิ และวิธีตัดสินเมื่อ annotator เห็นต่างกัน

3. Preprocessing และการสร้าง Dataset



ลำดับการประมวลผล

1. อ่านภาพและ annotation จาก manifest version ที่กำหนด
2. ตรวจสอบ resolution, aspect ratio และความถูกต้องของ polygon
3. Resize ภาพและ transform mask/bounding box ให้ตรงกัน
4. แปลง RGB เป็น HSV และใช้ Gaussian filter เพื่อลด noise
5. ทำ background subtraction เพื่อสร้าง fruit mask และ silhouette
6. ตรวจสอบคุณภาพ: mask ไม่หลุดกรอบ, contour หลักมีขนาดสมเหตุสมผล และ bounding box อยู่ภายในผลไม้
7. เก็บ output ใน `dataprocessed/` พร้อม preprocessing version

การแบ่งข้อมูล

Train	ฝึก weights และเรียนรู้ feature	ทำ augmentation ได้
Validation	เลือก threshold, checkpoint และ hyperparameters	ห้ามใช้ test แทน
Test	รายงานผลสุดท้าย	แข่งแข่งตั้งแต่ต้น และใช้ครั้งสุดท้าย

Quality gate: ต้องมีรายงานจำนวน “ผลไม้จริง” ต่อ split, distribution ของเกรด/ชนิดตำหนิ, duplicate check และ sample gallery ก่อน train

4. Baseline และการฝึกโมเดล

A. HSV Baseline

1. แปลง HSV
2. threshold สี/ความสว่าง
3. opening, closing, fill holes
4. หา contour และ defect-area ratio
5. เลือก threshold เกรดจาก validation

B. YOLOv8-nano

1. ใช้ defect bounding box
2. ฝึกตรวจตำแหน่งและ confidence
3. วัด IoU, precision, recall และ mAP

C. MobileNetV2 RGB-only

1. รับ fruit crop แบบ RGB
2. จำแนก 3 เกรด
3. เป็น baseline ของ CNN

D. MobileNetV2 Multi-input

1. branch หนึ่งรับ RGB
2. branch สองรับ silhouette
3. รวม feature แล้วจำแนก 3 เกรด
4. เปรียบกับ RGB-only

สิ่งที่ต้องบันทึกในทุก experiment

dataset_hash, split version, preprocessing version, random seed, input size, augmentation, class weighting, learning rate, checkpoint, model version และ hardware

สัญญาการเชื่อมโมเดล

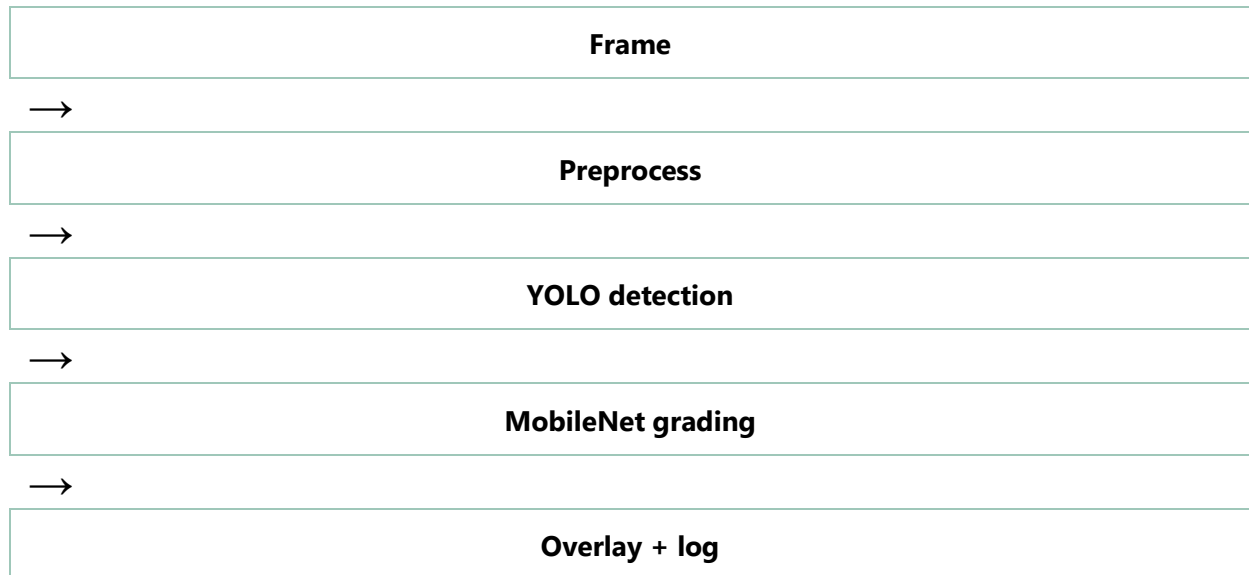
YOLOv8-nano รับผิดชอบการระบุตำแหน่งตำหนิ ส่วน MobileNetV2 รับผิดชอบเกรด โดยใช้ fruit crop และ ข้อมูลตำหนิที่ตรวจพบเป็น input ตาม policy เดียวกัน กรณี confidence ต่ำต้องมีสถานะ “ต้องตรวจซ้ำ” ไม่ควร บังคับให้ระบบตัดสินใจแบบไร้เงื่อนไข

5. Evaluation และ Real-time Inference

Grading	Accuracy, Precision, Recall, F1, Macro-F1	per-class metrics และ confusion matrix
Localization	IoU, precision, recall, mAP	ตำแหน่งผิด, missed defect และ small defect cases

Speed	Inference time, mean/p95 latency, FPS	capture, preprocess, detection, grading, rendering และ total
Ablation	HSV vs RGB vs RGB+silhouette	ผลบน test manifest เดียวกันและความแตกต่างของคะแนน

Inference flow



ผลลัพธ์ต่อ frame ต้องมี `defect_boxes/masks`, grade, confidence, timing, model version และ dataset version

เกณฑ์ 15 FPS: ต้องวัดแบบ end-to-end รวม preprocessing และ rendering บน hardware ที่ประกาศชัดเจน ไม่ใช่วัดเฉพาะเวลา forward pass ของโมเดล

6. Deployment, โครงสร้างไฟล์ และเกณฑ์รับมอบ

โครงสร้างที่แนะนำ

<code>data/raw/</code>	ภาพต้นฉบับแบบ immutable
<code>data/annotations/</code>	polygon, mask, bounding box, grade และ review metadata
<code>data/processed/</code>	RGB, HSV, silhouette และ transformed labels
<code>data/splits/</code>	manifest ของ train/validation/test

configs/, models/, reports/

การทดลอง, weights/export และรายงานผล

src/

ingestion, preprocessing, baseline, training, evaluation และ inference

ลำดับส่งมอบ

1. **Phase A:** data contract, label guide, ingestion และ split validation
2. **Phase B:** preprocessing QA และ HSV baseline
3. **Phase C:** YOLOv8-nano และ RGB-only MobileNetV2
4. **Phase D:** RGB+silhouette, ablation และ evaluation report
5. **Phase E:** unified inference, ONNX export และ benchmark

Acceptance criteria

- รันซ้ำจาก manifest/config เดิมได้ผลสอดคล้องกัน
- ไม่มี fruit_id ร่วข้าม split
- รายงาน metrics ครบทุกเกรตและ localization metrics
- ความแม่นยำถึงเกณฑ์ $\geq 85\%$ ตาม metric ที่ตกลง
- ความเร็ว end-to-end ถึง ≥ 15 FPS บน hardware ที่ระบุ
- ผ่านการตรวจ false positive, false negative และกรณีตำหนิเล็ก/ถูกบัง

สิ่งที่ยังต้องตัดสินใจก่อนเริ่มจริง: ชื่อเกรตและกฎตัดสินแบบเป็นทางการ, วิธี adjudication ระหว่าง annotator และ hardware ที่จะใช้ประกาศผล FPS

เอกสารนี้สรุป pipeline เชิงวิจัย ไม่รวมการประเมินคุณภาพภายในผลไม้, สภาพสวนจริง และการควบคุม servo/pneumatic ในระยะแรก